Module 3: Advanced Statistics for Machine Learning and AI

Comprehensive Guide for Data Science Professionals

Introduction

Statistics is the mathematical foundation of machine learning and artificial intelligence. Every machine learning model relies on statistical principles—from understanding data distributions to validating model performance. This module equips you with the statistical knowledge essential for building reliable, interpretable AI systems.

The journey through advanced statistics involves understanding how to characterize data (descriptive statistics), make inferences from samples (inferential statistics), model uncertainty (probability distributions), and test hypotheses. These concepts are not merely theoretical; they directly impact how you design experiments, validate models, and make data-driven decisions in production environments.

Whether you're A/B testing a recommendation system, validating that a model is statistically significant, or understanding the assumptions behind your algorithms, the statistical foundations covered in this module will prove invaluable throughout your career.

Part 1: Statistics in Data Science

What is Statistics?

Statistics is the science of collecting, analyzing, and interpreting data. It provides methodologies to extract meaningful insights from data and quantify uncertainty.

Two Main Branches:

1. **Descriptive Statistics** - Summarizing and describing data characteristics (mean, variance, distributions)
2. **Inferential Statistics** - Making predictions and generalizations about populations based on sample data

How Statistics is Used in Data Science

Application	Statistical Concept
Feature Selection	Correlation, p-values, feature importance
Model Validation	Cross-validation, confidence intervals, hypothesis tests

Application	Statistical Concept
Hypothesis Testing	A/B testing, statistical significance
Data Understanding	Distribution analysis, outlier detection
Performance Metrics	Precision, recall, AUC, R-squared
Uncertainty Quantification	Confidence intervals, prediction intervals

Population vs Sample

- **Population**: The entire set of items of interest (often infinite or impractical to measure)
- **Sample**: A subset of the population used to make inferences

Why Sampling? Testing an entire population is usually impossible or cost-prohibitive. Statistics helps us make accurate inferences from samples.

```
import numpy as np
import matplotlib.pyplot as plt

# Population: All test scores (hypothetically infinite)
population = np.random.normal(loc=70, scale=15, size=100000)

# Sample: Scores from 100 students
sample = np.random.choice(population, size=100)

print(f"Population mean: {population.mean():.2f}")
print(f"Sample mean: {sample.mean():.2f}")
print(f"Population std: {population.std():.2f}")
print(f"Sample std: {sample.std():.2f}")
```

Parameter vs Statistic

- **Parameter**: A characteristic of a population (e.g., population mean μ)
- **Statistic**: A characteristic of a sample (e.g., sample mean $\bar{x}$)

We use sample statistics to estimate population parameters.

Variables and Their Types

Type	Description	Examples
Categorical (Qualitative)	Categories with no inherent order	Gender, Color, Category
Ordinal	Categories with a natural order	Education level, Customer rating
Discrete Numerical	Countable values	Number of items, Age (in years)
Continuous Numerical	Any value within a range	Height, Temperature, Price

Part 2: Data Gathering Techniques

Data Collection Methods

1. **Primary Data**
   - **Survey**: Questionnaires, interviews
   - **Observation**: Direct observation, experiment
   - **Measurement**: Using instruments or sensors
2. **Secondary Data**
   - **Published Sources**: Books, journals, government databases
   - **Digital Sources**: APIs, public datasets, web scraping
   - **Internal Sources**: Company databases, transaction logs

Sampling Techniques

1. Convenience Sampling

Selecting easily accessible data. **Drawback**: Highly biased, not representative of population.

```
# Example: Surveying only customers visiting your store on a Saturday
# Problem: Misses customers who prefer to shop on weekdays
```

2. Simple Random Sampling

Every item in the population has an equal probability of selection. **Advantage**: Unbiased, statistically sound.

```
import random

population = list(range(1, 1001))  # Population: 1-1000
sample = random.sample(population, 100)  # Random sample of 100
print(sample)
```

3. Stratified Sampling

Dividing the population into strata (subgroups) and randomly sampling from each stratum. **Advantage**: Ensures representation from each subgroup.

```
import pandas as pd
import numpy as np

# Create population with different strata
population = pd.DataFrame({
    'Income': np.random.choice(['Low', 'Medium', 'High'], 1000),
    'Value': np.random.rand(1000)
```

```
})

# Stratified sampling
sample = population.groupby('Income', group_keys=False).apply(
    lambda x: x.sample(n=min(33, len(x))))
print(sample.head())
```

4. Systematic Sampling

Selecting every k-th item from a population. **Advantage**: Simple, systematic.

```
import numpy as np

population = np.arange(1, 1001)
k = 10  # Select every 10th item
sample = population[::k]
print(sample)
```

5. Cluster Sampling

Dividing the population into clusters and randomly selecting entire clusters. **Advantage**: Cost-effective for geographically dispersed populations.

```
# Example: Survey students in schools (clusters)
# Randomly select 5 schools, then survey all students in selected schools
```

Part 3: Descriptive Statistics

Measures of Central Tendency

Central tendency measures describe the "center" of a dataset.

Mean (Average)

The sum of all values divided by the number of values.

$$\mu = \frac{1}{n}\sum_{i=1}^{n} x_i$$

Pros: Uses all data, standard measure
Cons: Sensitive to outliers

```
import numpy as np

data = np.array([10, 20, 30, 40, 150])  # 150 is an outlier
print(f"Mean: {np.mean(data)}")  # 50 (affected by outlier)
```

Median

The middle value when data is sorted.

Pros: Robust to outliers
Cons: Doesn't use all information

```
import numpy as np

data = np.array([10, 20, 30, 40, 150])
print(f"Median: {np.median(data)}")  # 30 (not affected by outlier)
```

Mode

The most frequently occurring value.

Pros: Works for categorical data
Cons: May not exist or may be multiple

```
from scipy import stats

data = [1, 1, 2, 2, 2, 3, 4, 4, 4, 4, 5]
print(f"Mode: {stats.mode(data, keepdims=True)[^0]}")  # 4
```

Measures of Dispersion

Dispersion measures describe the "spread" of a dataset.

Range

Maximum value - Minimum value

```
import numpy as np

data = np.array([10, 20, 30, 40, 50])
print(f"Range: {np.max(data) - np.min(data)}")  # 40
```

Variance

Average of squared deviations from the mean.

$$\sigma^2 = \frac{1}{n}\sum_{i=1}^{n}(x_i - \mu)^2$$

```
import numpy as np
```

```
data = np.array([10, 20, 30, 40, 50])
print(f"Variance: {np.var(data)}")  # 200
```

Standard Deviation

Square root of variance. **Interpretation**: Average distance from the mean.

$$\sigma = \sqrt{\sigma^2}$$

```
import numpy as np

data = np.array([10, 20, 30, 40, 50])
print(f"Std Dev: {np.std(data)}")  # 14.14
```

Coefficient of Variation

Standard deviation divided by mean. **Use**: Compare spread across different scales.

$$CV = \frac{\sigma}{\mu} \times 100\%$$

```
import numpy as np

data1 = np.array([10, 20, 30, 40, 50])  # CV = 47%
data2 = np.array([100, 200, 300, 400, 500])  # CV = 47%

print(f"CV1: {(np.std(data1) / np.mean(data1)) * 100:.1f}%")
print(f"CV2: {(np.std(data2) / np.mean(data2)) * 100:.1f}%")
```

Skewness and Kurtosis

Skewness

Measures the asymmetry of a distribution.

- **Skewness = 0**: Symmetric distribution
- **Skewness > 0**: Right-skewed (tail on right)
- **Skewness < 0**: Left-skewed (tail on left)

```
from scipy import stats
import numpy as np

# Normal distribution
normal_data = np.random.normal(0, 1, 1000)
print(f"Normal Skewness: {stats.skew(normal_data):.3f}")

# Right-skewed distribution
```

```
right_skewed = np.random.chisquare(2, 1000)
print(f"Right-skewed Skewness: {stats.skew(right_skewed):.3f}")
```

Kurtosis

Measures the "tailedness" of a distribution.

- **Kurtosis = 3** (Excess = 0): Normal distribution
- **Kurtosis > 3** (Excess > 0): Heavy tails, peaked (leptokurtic)
- **Kurtosis < 3** (Excess < 0): Light tails, flat (platykurtic)

```
from scipy import stats
import numpy as np

normal_data = np.random.normal(0, 1, 1000)
print(f"Normal Kurtosis: {stats.kurtosis(normal_data, fisher=False):.3f}")
```

Box Plots and Outlier Detection

A box plot visualizes the distribution using quartiles:

- **Q1** (25th percentile): First quartile
- **Q2** (50th percentile): Median
- **Q3** (75th percentile): Third quartile
- **IQR** = Q3 - Q1: Interquartile range

Outliers: Values below Q1 - 1.5×IQR or above Q3 + 1.5×IQR

```
import numpy as np
import matplotlib.pyplot as plt

data = np.array([10, 15, 20, 25, 30, 35, 40, 45, 50, 100])  # 100 is outlier

Q1 = np.percentile(data, 25)
Q3 = np.percentile(data, 75)
IQR = Q3 - Q1

lower_bound = Q1 - 1.5 * IQR
upper_bound = Q3 + 1.5 * IQR

outliers = data[(data &lt; lower_bound) | (data &gt; upper_bound)]
print(f"Outliers: {outliers}")  # [^100]

plt.boxplot(data)
plt.ylabel("Value")
plt.title("Box Plot with Outlier Detection")
plt.show()
```

Covariance and Correlation

Covariance

Measures how two variables change together.

$$Cov(X, Y) = \frac{1}{n}\sum_{i=1}^{n}(x_i - \mu_x)(y_i - \mu_y)$$

- **Cov > 0**: Variables increase together
- **Cov < 0**: One increases, the other decreases
- **Cov ≈ 0**: No linear relationship

Correlation

Standardized covariance, ranges from -1 to +1. **Interpretation**:

- **r = +1**: Perfect positive correlation
- **r = 0**: No correlation
- **r = -1**: Perfect negative correlation

```
import numpy as np
import pandas as pd

x = np.array([1, 2, 3, 4, 5])
y = np.array([2, 4, 5, 4, 6])

# Using NumPy
correlation = np.corrcoef(x, y)[0, 1]
print(f"Correlation: {correlation:.3f}")

# Using Pandas
df = pd.DataFrame({'X': x, 'Y': y})
print(df.corr())

# Interpret: Positive correlation indicates variables move together
```

Part 4: Probability Distributions

Probability and Limitations

Probability quantifies uncertainty. It ranges from 0 (impossible) to 1 (certain).

Probability Rules:

- $P(A) + P(\neg A) = 1$
- $P(A \cup B) = P(A) + P(B) - P(A \cap B)$

- $P(A \cap B) = P(A) \times P(B)$ (if independent)

Limitations:

- Past probabilities don't guarantee future outcomes
- Assumes stable conditions
- Model assumptions may not hold in reality

Discrete Probability Distributions

Discrete distributions model outcomes that can take only specific values.

Bernoulli Distribution

Outcomes: Success (1) or Failure (0). Parameter: p (probability of success)

```
from scipy.stats import bernoulli
import matplotlib.pyplot as plt

p = 0.7  # Probability of success
rv = bernoulli(p)

# Generate samples
samples = rv.rvs(1000)
print(f"Mean: {samples.mean():.2f} (Expected: {p})")

# Probability distribution
plt.bar([0, 1], [1-p, p])
plt.xlabel("Outcome")
plt.ylabel("Probability")
plt.title("Bernoulli Distribution (p=0.7)")
plt.show()
```

Binomial Distribution

Number of successes in n independent Bernoulli trials.

$$P(X = k) = \binom{n}{k} p^k (1-p)^{n-k}$$

Parameters: n (number of trials), p (success probability)

Example: Number of heads in 10 coin flips

```
from scipy.stats import binom
import matplotlib.pyplot as plt

n = 10  # Number of trials
p = 0.5  # Probability of success

# Probability of exactly k successes
```

```
x = range(0, n+1)
prob = binom.pmf(x, n, p)

plt.bar(x, prob)
plt.xlabel("Number of Successes")
plt.ylabel("Probability")
plt.title(f"Binomial Distribution (n={n}, p={p})")
plt.show()

# Example: Probability of exactly 6 heads
print(f"P(X=6): {binom.pmf(6, n, p):.4f}")

# Cumulative probability
print(f"P(X≤6): {binom.cdf(6, n, p):.4f}")
```

Poisson Distribution

Number of events in a fixed interval. **Parameter**: λ (mean rate)

Example: Number of emails received in an hour

```
from scipy.stats import poisson
import matplotlib.pyplot as plt

lambda_ = 3  # Average events per interval

x = range(0, 10)
prob = poisson.pmf(x, lambda_)

plt.bar(x, prob)
plt.xlabel("Number of Events")
plt.ylabel("Probability")
plt.title(f"Poisson Distribution (λ={lambda_})")
plt.show()

# Example: Probability of exactly 5 events
print(f"P(X=5): {poisson.pmf(5, lambda_):.4f}")
```

Continuous Probability Distributions

Continuous distributions model outcomes that can take any value within a range.

Normal Distribution

The most important distribution in statistics. Characterized by mean (μ) and standard deviation (σ).

Properties:

- Bell-shaped, symmetric around mean
- 68% of data within ±1σ, 95% within ±2σ, 99.7% within ±3σ
- Central Limit Theorem: Sample means approach normal distribution

```
from scipy.stats import norm
import numpy as np
import matplotlib.pyplot as plt

mu = 100  # Mean
sigma = 15  # Standard deviation

# Generate data
data = np.random.normal(mu, sigma, 10000)

# Plot histogram
plt.hist(data, bins=50, density=True, alpha=0.7)

# Plot theoretical distribution
x = np.linspace(mu - 4*sigma, mu + 4*sigma, 100)
plt.plot(x, norm.pdf(x, mu, sigma), 'r-', linewidth=2)
plt.xlabel("Value")
plt.ylabel("Probability Density")
plt.title(f"Normal Distribution (μ={mu}, σ={sigma})")
plt.show()

# Probability calculations
print(f"P(X &lt; 115): {norm.cdf(115, mu, sigma):.4f}")
print(f"P(85 &lt; X &lt; 115): {norm.cdf(115, mu, sigma) - norm.cdf(85, mu, sigma):.4f}")
```

Standard Normal Distribution

Normal distribution with μ = 0 and σ = 1. Used for standardizing values.

$$Z = \frac{X - \mu}{\sigma}$$

Significance: Z-scores allow comparison across different scales.

```
# Convert data to z-scores
data = np.array([85, 90, 100, 110, 115])
z_scores = (data - data.mean()) / data.std()
print(z_scores)  # [-1.41, -0.71, 0, 0.71, 1.41]
```

Part 5: Inferential Statistics

Sampling Variability and Central Limit Theorem

Sampling Variability: Different samples from the same population produce different statistics.

Central Limit Theorem (CLT): For any population distribution, the distribution of sample means approaches a normal distribution as sample size increases, regardless of the population distribution.

```
import numpy as np
import matplotlib.pyplot as plt
```

```
# Population: Uniform distribution (not normal)
population = np.random.uniform(0, 100, 100000)

# Draw many samples and calculate their means
sample_means = []
for _ in range(1000):
    sample = np.random.choice(population, size=30)
    sample_means.append(np.mean(sample))

# Plot sample means
plt.hist(sample_means, bins=30, density=True)
plt.title("Distribution of Sample Means (CLT)")
plt.xlabel("Sample Mean")
plt.ylabel("Frequency")
plt.show()

# Note: Despite population being uniform, sample means are approximately normal
```

Confidence Intervals

A **confidence interval** provides a range of plausible values for a population parameter.

Interpretation: 95% CI means if we repeated sampling 100 times, approximately 95 intervals would contain the true parameter.

Confidence Interval for Mean

$$CI = \bar{x} \pm z^* \times \frac{\sigma}{\sqrt{n}}$$

Where z^* depends on confidence level (1.96 for 95%)

```
import numpy as np
from scipy import stats

data = np.array([100, 105, 98, 102, 101, 99, 103, 97, 104, 101])

sample_mean = np.mean(data)
sample_std = np.std(data, ddof=1)
n = len(data)

# 95% Confidence Interval
z_critical = 1.96
margin_error = z_critical * (sample_std / np.sqrt(n))
ci_lower = sample_mean - margin_error
ci_upper = sample_mean + margin_error

print(f"95% CI: [{ci_lower:.2f}, {ci_upper:.2f}]")
print(f"Interpretation: We're 95% confident the true mean is between {ci_lower:.2f} and {

# Using scipy
```

```
ci = stats.t.interval(0.95, len(data)-1, loc=sample_mean, scale=stats.sem(data))
print(f"Using scipy: {ci}")
```

Hypothesis Testing

Hypothesis Testing determines whether sample evidence supports a claim about a population.

Steps:

1. State null and alternative hypotheses
2. Choose significance level (α)
3. Calculate test statistic
4. Determine p-value
5. Make decision (reject or fail to reject null hypothesis)

Null Hypothesis (H_0): Statement of no effect or no difference
Alternative Hypothesis (H_1): What we're trying to prove

```
import numpy as np
from scipy import stats

# Scenario: Testing if average product rating is 4.0
data = np.array([3.8, 4.1, 3.9, 4.2, 3.7, 4.0, 3.9, 4.1])

# H0: μ = 4.0
# H1: μ ≠ 4.0 (two-tailed)

sample_mean = np.mean(data)
sample_std = np.std(data, ddof=1)
n = len(data)
population_mean = 4.0

# Calculate t-statistic
t_statistic = (sample_mean - population_mean) / (sample_std / np.sqrt(n))

# Calculate p-value
p_value = 2 * (1 - stats.t.cdf(abs(t_statistic), n-1))

print(f"t-statistic: {t_statistic:.4f}")
print(f"p-value: {p_value:.4f}")

if p_value &lt; 0.05:
    print("Reject H0: Evidence that mean rating differs from 4.0")
else:
    print("Fail to reject H0: Insufficient evidence that mean rating differs")
```

Z-Test and T-Test

Z-Test

Used when:

- Population standard deviation is known
- Sample size > 30
- Population is normal

```
from scipy import stats
import numpy as np

# Sample data
sample = np.array([102, 105, 98, 101, 103, 99, 100, 104])
population_mean = 100
population_std = 2

# Z-test
z_statistic = (np.mean(sample) - population_mean) / (population_std / np.sqrt(len(sample)
p_value = 2 * (1 - stats.norm.cdf(abs(z_statistic)))

print(f"Z-statistic: {z_statistic:.4f}")
print(f"p-value: {p_value:.4f}")
```

T-Test

Used when:

- Population standard deviation is unknown
- Sample size < 30 or unknown
- Population approximately normal

Types:

- **One-sample t-test**: Compare sample mean to population mean
- **Two-sample t-test**: Compare two sample means
- **Paired t-test**: Compare matched pairs

```
from scipy import stats
import numpy as np

# One-sample t-test
sample = np.array([102, 105, 98, 101, 103, 99, 100, 104])
t_statistic, p_value = stats.ttest_1samp(sample, popmean=100)
print(f"T-statistic: {t_statistic:.4f}, p-value: {p_value:.4f}")

# Two-sample t-test
group1 = np.array([85, 90, 92, 88, 87])
group2 = np.array([78, 82, 80, 81, 79])
```

```
t_statistic, p_value = stats.ttest_ind(group1, group2)
print(f"T-statistic: {t_statistic:.4f}, p-value: {p_value:.4f}")

# Paired t-test (before and after)
before = np.array([100, 102, 98, 101, 99])
after = np.array([105, 110, 103, 108, 102])
t_statistic, p_value = stats.ttest_rel(before, after)
print(f"T-statistic: {t_statistic:.4f}, p-value: {p_value:.4f}")
```

Chi-Square Test

Used for testing independence between categorical variables.

Null Hypothesis: Two categorical variables are independent
Alternative Hypothesis: Variables are associated

```
from scipy import stats
import numpy as np

# Contingency table: Gender vs Product Preference
# Rows: Male, Female; Columns: Product A, Product B, Product C
contingency_table = np.array([[50, 40, 30],
                              [30, 50, 40]])

chi2, p_value, dof, expected = stats.chi2_contingency(contingency_table)

print(f"Chi-square statistic: {chi2:.4f}")
print(f"p-value: {p_value:.4f}")
print(f"Degrees of freedom: {dof}")
print(f"Expected frequencies:\n{expected}")

if p_value &lt; 0.05:
    print("Reject H0: Gender and product preference are associated")
else:
    print("Fail to reject H0: No significant association")
```

F-Test and ANOVA

ANOVA (Analysis of Variance) tests if means of 3+ groups are equal.

```
from scipy import stats
import numpy as np

# Three groups of test scores
group1 = np.array([85, 90, 92, 88, 87])
group2 = np.array([78, 82, 80, 81, 79])
group3 = np.array([92, 95, 93, 96, 91])

# ANOVA F-test
f_statistic, p_value = stats.f_oneway(group1, group2, group3)

print(f"F-statistic: {f_statistic:.4f}")
```

```
print(f"p-value: {p_value:.4f}")

if p_value &lt; 0.05:
    print("Reject H0: At least one group mean differs significantly")
else:
    print("Fail to reject H0: No significant difference between groups")
```

Part 6: Common Mistakes and Best Practices

Statistical Pitfalls

1. **p-hacking**: Repeatedly testing hypotheses until significant results are found
2. **Multiple Testing Problem**: Conducting many tests without correcting significance level
3. **Correlation ≠ Causation**: Correlation doesn't imply causal relationship
4. **Ignoring Effect Size**: p-value alone doesn't tell the magnitude of effect
5. **Misinterpreting Confidence Intervals**: Not probability about parameter given data

Best Practices

- **Define hypotheses before analysis**: Prevent p-hacking
- **Use appropriate sample size**: Power analysis ensures adequate sample
- **Check assumptions**: Verify normality, homogeneity of variance, etc.
- **Report effect sizes**: Not just p-values
- **Visualize distributions**: Catch unusual patterns
- **Document methods**: Enable reproducibility
- **Consider practical significance**: Statistical significance ≠ practical importance

Real-World Case Study: A/B Testing

Scenario

An e-commerce company tests whether a new checkout button color increases conversion rates.

```
import numpy as np
from scipy import stats

# Data collection
control_group = np.array([0, 1, 0, 1, 1, 0, 1, 1, 0, 1])  # 1 = converted
treatment_group = np.array([1, 1, 0, 1, 1, 1, 1, 0, 1, 1])  # 1 = converted

control_rate = np.mean(control_group)
treatment_rate = np.mean(treatment_group)

print(f"Control conversion rate: {control_rate:.1%}")
```

```
print(f"Treatment conversion rate: {treatment_rate:.1%}")

# Hypothesis test
chi2, p_value, dof, expected = stats.chi2_contingency(
    np.array([[sum(control_group), len(control_group) - sum(control_group)],
              [sum(treatment_group), len(treatment_group) - sum(treatment_group)]]))

print(f"\np-value: {p_value:.4f}")

if p_value &lt; 0.05:
    print("Statistically significant difference detected!")
else:
    print("No statistically significant difference (yet)")

# Effect size (absolute uplift)
uplift = treatment_rate - control_rate
print(f"Absolute uplift: {uplift:.1%}")
```

References and Further Learning

1. Wasserman, L. (2023). All of Statistics: A Concise Course in Statistical Inference (2nd ed.). Springer.
2. Casella, G., & Berger, R. L. (2024). Statistical Inference (3rd ed.). Cengage Learning.
3. Freedman, D., Pisani, R., & Purves, R. (2023). Statistics (4th ed.). W.W. Norton & Company.
4. Hastie, T., Tibshirani, R., & James, G. (2023). An Introduction to Statistical Learning (2nd ed.). Springer.
5. McElreath, R. (2023). Statistical Rethinking: A Bayesian Course with Examples in R and Stan (2nd ed.). Chapman & Hall/CRC.
6. Kanjilal, S. (2024). Applied Statistics for Machine Learning. DataCamp Blog.
7. StatQuest with Josh Starmer. (2024). Statistics Fundamentals. Retrieved from https://www.youtube.com/
8. SciPy Documentation. (2024). Statistical Functions. Retrieved from https://docs.scipy.org/
9. Kaggle Learn. (2024). Statistical Thinking in Python. Retrieved from https://www.kaggle.com/
10. Cohen, J. (2023). Statistical Power Analysis for the Behavioral Sciences (3rd ed.). Lawrence Erlbaum.

Key Takeaways

- **Statistics is the language of data**: Every ML model relies on statistical principles
- **Sample statistics estimate population parameters**: Sampling enables practical inference
- **Distributions model uncertainty**: Understanding probability distributions is foundational
- **Hypothesis testing validates claims**: Rigorous testing prevents false discoveries
- **Statistical significance ≠ practical importance**: Always consider effect size and context

- **Assumptions matter**: Violating test assumptions invalidates results
- **Visualization reveals patterns**: Always plot your data before analysis
- **Replication builds confidence**: Reproducibility is essential in science

Career Application

As an AI/ML professional, you'll use statistics to:

1. **Design experiments**: Determine sample sizes for A/B tests
2. **Validate models**: Use statistical tests to verify model performance
3. **Debug models**: Identify statistical issues causing poor predictions
4. **Communicate results**: Use confidence intervals and effect sizes to stakeholders
5. **Make decisions**: Apply statistical reasoning to business problems
6. **Avoid pitfalls**: Recognize and prevent common statistical mistakes

Master these concepts and you'll be equipped to build reliable, interpretable, and trustworthy AI systems.
[1] [2] [3] [4] [5] [6] [7] [8] [9] [10] [11] [12] [13] [14] [15] [16] [17] [18] [19] [20] [21] [22] [23] [24] [25] [26] [27] [28] [29] [30]

⁂

1. https://discuss.scientific-python.org/t/best-practices-for-optimizing-numpy-performance-in-scientific-python-projects/1621
2. https://pyseek.com/2024/09/seaborn-in-python/
3. https://www.investopedia.com/terms/d/descriptive_statistics.asp
4. https://www.almabetter.com/bytes/tutorials/gate-data-science-and-ai/descriptive-statistics-for-gate
5. https://economia.uniroma2.it/ba/globalgovernance/corso/asset/YTo0OntzOjI6ImlkIjtzOjQ6IjI1ODYiO3M6MzoiaWRhIjtzOjU6Ijc0MDI5IjtzOjI6ImVtIjtOO3M6MToiYyI7czo1OiIxNmI1NyI7fQ==
6. https://ledidi.com/academy/measures-of-central-tendency-mean-median-and-mode
7. https://www.dissertationindia.com/blog/post/what-is-descriptive-statistics-and-how-to-use-it
8. https://www.geeksforgeeks.org/maths/binomial-distribution/
9. https://www.scribd.com/document/589259400/Test-of-Hypothesis-t-and-Z-tests-Chi-square-Test-F-Test
10. https://www.simplilearn.com/what-is-descriptive-statistics-article
11. https://www.healthknowledge.org.uk/public-health-textbook/research-methods/1b-statistical-methods/statistical-distributions
12. https://www.cybrosys.com/blog/python-numpy-tutorial-2024
13. https://www.datacamp.com/tutorial/t-test-vs-z-test
14. https://serpapi.com/blog/beautiful-soup-build-a-web-scraper-with-python/
15. https://realpython.com/beautiful-soup-web-scraper-python/
16. https://www.browserstack.com/guide/web-scraping-using-beautiful-soup
17. https://www.scrapingbee.com/blog/web-scraping-101-with-python/
18. https://www.scraperapi.com/web-scraping/beautiful-soup/

19. https://emeritus.org/in/learn/exploratory-data-analysis/
20. https://developers.google.com/edu/python/regular-expressions
21. https://www.promptcloud.com/blog/web-scraping-with-python/
22. https://www.linkedin.com/pulse/exploratory-data-analysis-eda-case-study-archana-khewariya-jqquf
23. https://expediteinformatics.com/exploring-numpy-in-2024-trends-standards-benefits-challenges-and-commitments/
24. https://www.geeksforgeeks.org/python/pattern-matching-python-regex/
25. https://www.projectpro.io/article/numpy-cheatsheet/922
26. https://realpython.com/numpy-example/
27. https://www.dasca.org/world-of-data-science/article/unlocking-data-manipulation-in-python-key-pandas-techniques
28. https://www.analytixlabs.co.in/blog/python-visualization/
29. https://numpy.org/doc/2.2/user/absolute_beginners.html
30. https://codebasics.io/blog/mastering-data-manipulation-with-pandas-a-comprehensive-guide-for-aspiring-data-scientists